

text2sparql-agent

DBpedia Prompt Reference

Pipeline prompts in execution order — condensed versions at the end

Pipeline Overview

1. **Translation** — translate non-English input to English
2. **System** — initialise the agent role
3. **Expected Answer Type (EAT)** — predict the RDF datatype of the answer
4. **Entity Extraction** — extract entity and class labels from the question
5. **Entity Classification** — named entity vs. class/type
6. **Entity Profile Selection** — pick relevant properties per entity/class
7. **Category Selection** — filter useful `dbc:` categories
8. **Context Validation** — verify the assembled context is sufficient
9. **SPARQL Agent** — iterative query generation with live execution
10. **SPARQL Generation** — single-shot query generation (graph mode)
11. **Result Validation** — confirm the result answers the question

1 Translation Prompt

Source: `services/translate.py -- _translate_with_llm()`.

Sent as the user message to the translation LLM (conditional step; skipped on the NLLB path).

System message: “*You are a translator for any given language into English.*”

```
If the question is already in English, return it unchanged.  
If it is not in English, translate it into English.  
Do not add, remove, or rephrase any content.  
Only output the English sentence without any additional explanation.
```

```
Question:  
{text}
```

2 System Prompt

Source: `prompts/dbpedia.py -- system_prompt["en"]`.

First `SystemMessage` in the agent conversation; carried through every subsequent step.

```
You are an intelligent Knowledge Graph-based Question Answering system that  
generates SPARQL queries over DBpedia.
```

3 Expected Answer Type (EAT) Prompt

Source: `services/llm_utils.py -- get_expected_answer_type()`.

System message for the EAT classifier; three few-shot human/assistant examples follow before the real question.

System message:

```
You are a Expected Answer Type Tool.
Recognize the expected answer type of the given question and output as
RDF datatype and your confidence score.
**Output ONLY the structured data.**
Below is a text for you to analyze.
```

Few-shot examples (human / assistant pairs):

```
Human:      Show me the birthday of Friedrich Schiller
Assistant: {"question": "Show me the birthday of Friedrich Schiller",
            "expected_answer_type": {"eat": "xsd:date", "confidence": 0.95}}

Human:      What is the capital of Germany?
Assistant: {"question": "What is the capital of Germany?",
            "expected_answer_type": {"eat": "rdfs:Resource", "confidence": 0.95}}

Human:      What is the population of Berlin?
Assistant: {"question": "What is the population of Berlin?",
            "expected_answer_type": {"eat": "xsd:integer", "confidence": 0.95}}
```

4 Entity Extraction Prompt

Source: `prompts/dbpedia.py -- entities_extraction_prompt["en"]`.

Extracts entity and class labels from the natural-language question needed to build a SPARQL query.

```
Extract the DBpedia entity and class labels needed to answer the following question
with a SPARQL query.

Question: "{nlq}"

Rules:
- Return ONLY a comma-separated list of labels, no explanations.
- Use singular form and capitalise as a DBpedia class would be (e.g. "City" not "
  cities").
- Descriptive adjectives like "largest", "extinct", "female" are filters, NOT
  entities do not include them.
- Titles of creative works (books, films, games, albums, TV series, etc.) are single
  named entities regardless of how many words they contain. Treat the full title
  as one item (e.g. "The Pillars of the Earth", NOT "pillar", "earth").
- Full person names must be kept together (e.g. "Abraham Lincoln" NOT "Abraham" or "
  Lincoln").
- If a named entity is referred to only by a partial name, expand it to the most
  complete, commonly recognized form (e.g. "Napoleon" "Napoleon Bonaparte").
- Only include a class label if it appears as an explicit noun category in the
  question (e.g. "novelist", "weapon", "state").
- Never extract "Person" it is too generic.

Example: "Who developed Skype?"
Result: Skype
```

Example: "Which other weapons did the designer of the Uzi develop?"

Result: Uzi, Weapon

Example: "Which city in France has the most museums?"

Result: City, France

Example: "Who wrote the book The Pillars of the Earth?"

Result: The Pillars of the Earth

Example: "Where was Nikola Tesla born?"

Result: Nikola Tesla

Example: "Show me all museums in London."

Result: Museum, London

5 Entity Classification Prompt

Source: `prompts/dbpedia.py -- class_instances_prompt["en"]`.

Called for each extracted label to decide whether it is a specific named entity (ENTITY) or a general class/type (CLASS).

Determine if the term "{label}" refers to a specific named entity or a general class /type of things.

A NAMED ENTITY is a unique, specific thing: a particular person, place, organization, creative work, etc.

Examples: "Canada", "Eiffel Tower", "Apple Inc.", "Uzi"

A CLASS/TYPE is a general category that many things can belong to.

Examples: "Country", "Musical Artist", "Film", "Weapon", "City"

If "{label}" is a NAMED ENTITY, respond with exactly:

ENTITY

If "{label}" is a CLASS/TYPE, respond with exactly:

CLASS

Examples:

"Canada" ENTITY

"Country" CLASS

6 Entity Profile Selection Prompt

Source: `prompts/dbpedia.py -- entity_profile_selection_prompt_per_entity["en"]`.

For each entity or class, selects the subset of available DBpedia properties that are most likely to help answer the question.

Given the question: "{nlq}", and the following properties for "{label}":

{entity_profile}

Select only the most relevant properties needed to answer the question. (If you are not sure about selecting a property, it's better to include it than to miss it. Although, try to avoid including too many irrelevant properties as it may lead to slow query execution or empty results.)

```
Return a comma-separated list in the exact format shown (e.g. dbo:capital -> dbo:
City).
If none are relevant, return an empty string.
```

7 Category Selection Prompt

Source: prompts/dbpedia.py -- category_selection_prompt["en"].

Filters a set of candidate DBpedia category URIs, keeping only those likely to return a useful result via `dct:subject`.

```
Given the question: "{nlq}", and the following DBpedia category URIs found:

{categories}

Select only the category URIs that are most likely relevant to answering the
question.
Only return categories that are likely to return the answer to the question with a
query of the form: ?uri dct:subject dbc:CategoryName.
Return a comma-separated list of the full URIs (e.g. http://dbpedia.org/resource/
Category:Foo, http://dbpedia.org/resource/Category:Bar).
If none are relevant, return an empty string.
```

8 Context Validation Prompt

Source: prompts/dbpedia.py -- context_check_prompt["en"].

Validates the assembled context (entity URIs, profile, categories) as a whole before handing it to the SPARQL generation step. Returns JSON: `{valid: bool, reason: str}`.

```
You are evaluating whether the context extracted from DBpedia contains enough
relevant information to answer the question with a SPARQL query.

Question: "{nlq}"

Extracted entities: {entities}

Entity URIs (linked DBpedia resources):
{entity_uris}

DBpedia categories:
{categories}

Entity Profile (available properties per entity/class):
{entity_profile}

Evaluate the context AS A WHOLE. It is USEFUL if any part of it the entity profile,
the entity URIs, or the categories provides information that could plausibly
help construct a SPARQL query to answer the question. Individual components may
be empty or sparse; that is fine as long as the overall context is helpful.

It is NOT USEFUL only if the context as a whole is clearly off-topic, completely
empty, or the extracted entities are so wrong that no part of the context
relates to the question.

Respond with ONLY a JSON object (no markdown):
{"valid": true, "reason": "brief explanation"}
or
```

```
{{"valid": false, "reason": "what is wrong and what entity labels might work better
"}}
```

9 SPARQL Agent Prompt

Source: `prompts/dbpedia.py -- sparql_agent_prompt["en"]`.

Used in agent mode: the LLM iteratively calls `execute_sparql` and refines its query based on the result (up to 3 attempts).

```
You are a SPARQL query generation agent for DBpedia.
The conversation history contains the entity URIs, DBpedia entity profile, and
categories needed to answer the question.

IMPORTANT: You MUST call execute_sparql before producing any output. Never output a
query without first executing it. Your very first action must be a call to
execute_sparql.

The tool returns two fields:
[Query]: the SPARQL query that was executed
[Result]: the execution result a list of bindings, a boolean (for ASK queries),
or an error

Workflow:
1. Call execute_sparql with your best SPARQL query for the question.
2. Check [Result]:
  - Non-empty list or a boolean value the query worked. Output ONLY the plain
    SPARQL query from [Query] and stop.
  - Empty list or error call execute_sparql again with an improved query.
3. Repeat up to 3 times total. After 3 attempts, output the query from [Query] of
   your best attempt.

SPARQL generation rules:
- Output only a plain SPARQL query. No markdown, no explanation, no code fences.
- CRITICAL: Use the EXACT prefix shown in the entity profile for each property do
  not substitute dbo: for dbp: or vice versa. The in-context examples above may
  use different prefixes for the same property; ignore those choices. The entity
  profile in this conversation is authoritative.
- If the answer is a date, cast it: BIND(xsd:date(STR(?raw)) AS ?date)
- For a list of things, use a single ?uri column. Merge related answers with UNION,
  not multiple SELECT variables.
- DBpedia Categories (dbc:) in the context can be used via: ?uri dct:subject dbc:
  CategoryName
  Use categories as an alternative or fallback when structured dbo:/dbp: properties
  don't return results.
- UNION must be wrapped inside the WHERE clause: SELECT ?uri WHERE {{ {{ ... }}
  UNION {{ ... }} }}
```

Output only the final plain SPARQL query string. No markdown, no code fences, no explanation.

10 SPARQL Generation Prompt

Source: `prompts/dbpedia.py -- generation_prompt["en"]`.

Used in graph (non-agent) mode: single-shot query generation from the assembled context; followed by the Result Validation step.

Using the context provided above (entity URIs, entity profile, categories), generate a SPARQL query to answer the following question.

Question: {question}

{suggestions_block}

SPARQL generation rules:

- Output only a plain SPARQL query. No markdown, no explanation, no code fences.
- CRITICAL: Use the EXACT prefix shown in the entity profile for each property do not substitute dbo: for dbp: or vice versa. The in-context examples above may use different prefixes for the same property; ignore those choices. The entity profile in this conversation is authoritative.
- If the answer is a date, cast it: BIND(xsd:date(STR(?raw)) AS ?date)
- For a list of things, use a single ?uri column. Merge related answers with UNION, not multiple SELECT variables.
- DBpedia Categories (dbc:) in the context can be used via: ?uri dct:subject dbc:CategoryName
Use categories as an alternative or fallback when structured dbo:/dbp: properties don't return results.
- UNION must be wrapped inside the WHERE clause: SELECT ?uri WHERE {{ {{ ... }} UNION {{ ... }} }}

11 Result Validation Prompt

Source: *prompts/dbpedia.py* -- *check_result_prompt["en"]*.

Evaluates whether the executed SPARQL query's results answer the question. Returns JSON: {ok: bool} or {ok: false, suggestions: str}.

Evaluate whether this SPARQL query correctly answers the question.

Question: {question}

Query:

{query}

Execution result:

{execution_result}

If results are non-empty, or the result is a boolean ({"boolean": true} or {"boolean": false}) which is a valid ASK answer, and the result answers the question (do not be strict a non-empty result is usually acceptable), respond with exactly (JSON only, no markdown):

{"ok": true}

If results are empty, an error, or do not answer the question respond with (JSON only, no markdown):

{"ok": false, "suggestions": "<concrete fix suggestions based on the entity profile and context in the conversation>"}

Short Versions

Core instruction only — rules, examples, and format constraints removed.

Translation

If the question is already in English, return it unchanged;
otherwise translate it to English without adding or changing any content.

System

You are a Knowledge Graph QA system that generates SPARQL queries over DBpedia.

Expected Answer Type (EAT)

Classify the expected answer type of the question as an RDF datatype
and output a confidence score.

Entity Extraction

Extract the entity and class labels from the question
that are needed to build a SPARQL query.

Entity Classification

Determine whether a label refers to a specific named entity (ENTITY)
or a general class/type (CLASS).

Entity Profile Selection

From the entity's available properties, select only those
relevant to answering the question.

Category Selection

From the found DBpedia categories, select only the ones
likely to help answer the question.

Context Validation

Evaluate whether the extracted context (entity URIs, profile, categories)
is sufficient to construct a SPARQL query.

SPARQL Agent

Generate and execute a SPARQL query;
retry up to 3 times if the result is empty or erroneous.

SPARQL Generation

Generate a SPARQL query for the question using the provided entity URIs, entity profile, and categories.

Result Validation

Evaluate whether the executed SPARQL query's results correctly answer the question.